



KAZAKH-BRITISH
TECHNICAL
UNIVERSITY

Programming Principles I

Lecture 1



Zhiger Bolatov

Senior Lecturer
School of IT&I

- What is programming?
- Code structure
- Programming environment
- Variables
- Data type (int, float, double, char, string, bool, void)
- Variable declarations
- Constant declarations
- Operators and Expressions
- Arithmetic operations
- Assignment (=) Operator
- C++ Shorthands
- ASCII table
- Math functions (sqrt, abs, sin, max, min, pow)
- Comments

What is Programming?

Programming is the process of creating instructions that tell a computer what to do.

A program:

- takes input from the user or other sources;
- processes the data;
- produces an output.

```
Input → Process → Output
5      → × 2      → 10
```

Code Structure

1	<i>#include <iostream></i>	Header File
2	<i>using namespace std;</i>	Namespace declaration
3	<i>int main()</i>	Execution begins at Main
4	<i>{</i>	
5	<i>// This prints Hello World!</i>	Comment
6	<i>cout << "Hello, World!";</i>	Statement
7	<i>return 0;</i>	Return
8	<i>}</i>	

A programming environment is a set of tools used to write, compile, run, and debug programs.

Main components

- Code Editor / IDE — write and edit source code
- Compiler — translates source code into machine code
- Debugger — helps find and fix errors
- Terminal / Console — run programs and interact with them

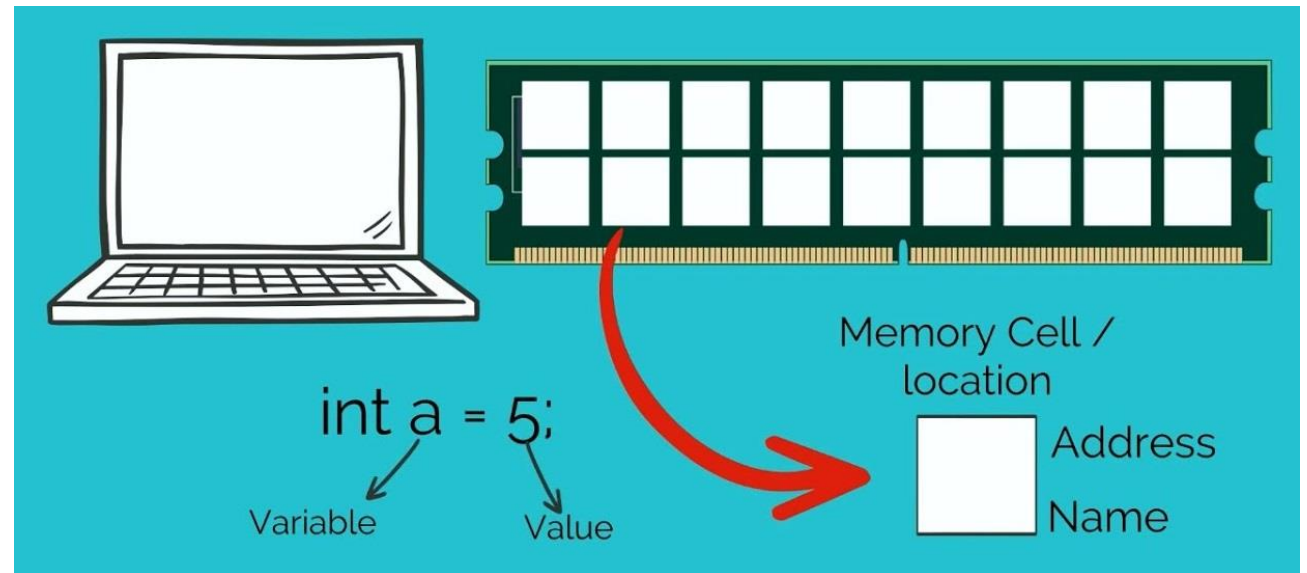
Example: (VS Code) + (C++ Compiler (GCC/Clang)) + (Terminal)

For more information, see:

https://github.com/ZhigerBolatov/PP1_2026

Variables

- A variable is a memory location that is used to store values.
- Consider the memory of the computer to contain of equal sized cells capable of storing data.
- These cells have unique addresses. Since we store data in these we need to refer to them in our program.
- It is humanly impossible to remember all the addresses.
- In order to be able to use the memory location as a variable we need to declare it ie allocate memory to it.
- A variable can be declared using a data type and a valid identifier.

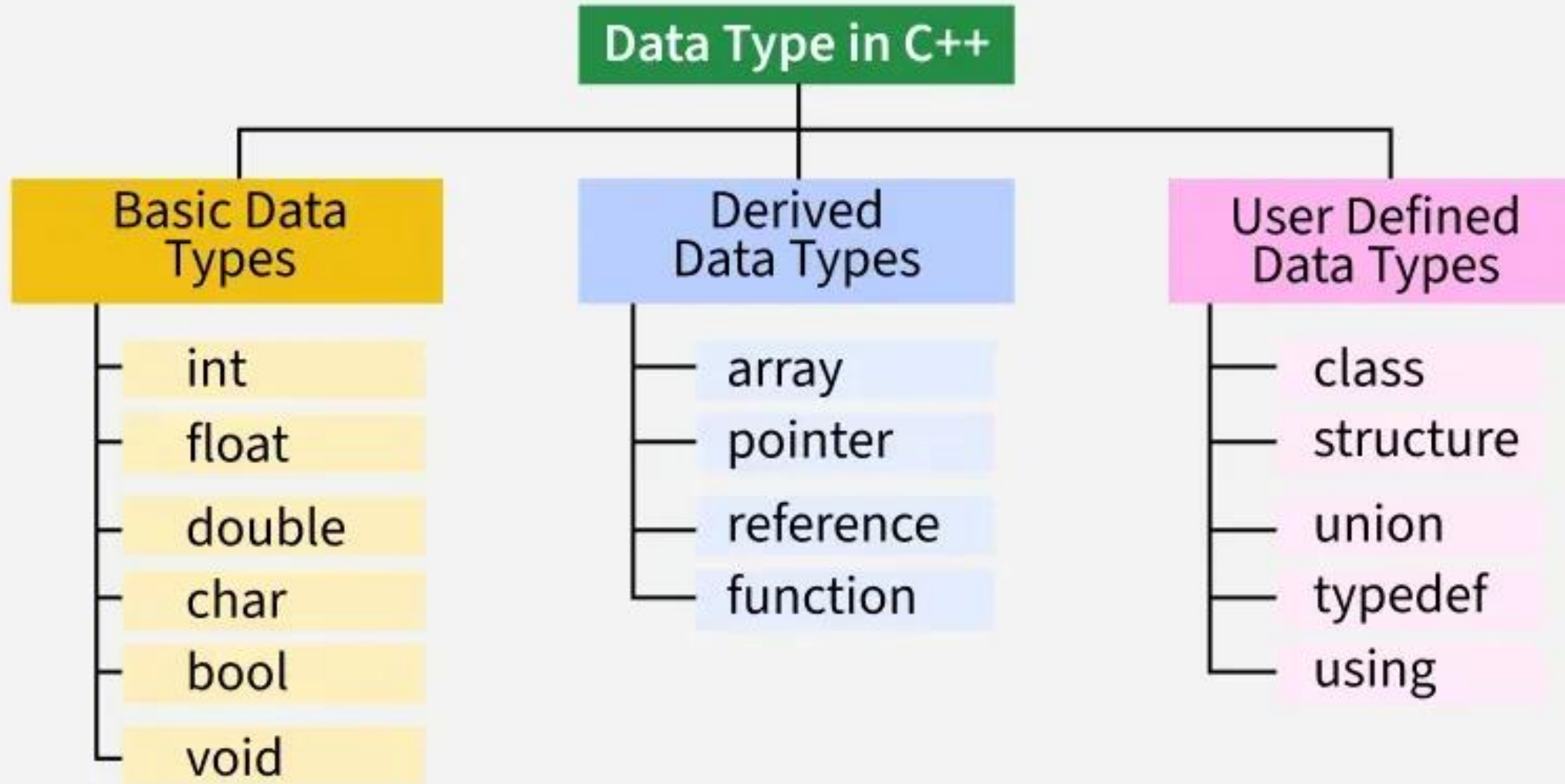


A data type defines a set of values and the operations that can be performed on that data.



As all of you must be aware that a computer is just a machine. It cannot by itself distinguish between various types of data. This means that it cannot distinguish between the number 20 and the letter 'A' or the word "good". For the computer all of this is just a piece of data. By using data types we can classify the different data for the computer so that it can be stored and processed in a certain manner.

Data Type



Data Type: int

Variables declared as integers are capable of storing whole numbers, these are numbers without decimal point.

There are three variants of integer type. These differ in size.

The following is a table showing the size and domain of integer data types.

Modifier	Size (bytes)	Min value	Max value	Sample data (To Store)
Signed	2	-32768	32767	Marks
Unsigned		0	65535	Age
Signed	2	-32768	32767	
Unsigned		0	65535	
Signed	4	-2,147,483,648	2,147,483,647	Population
Unsigned		0	4,294,967,295	

Data Type: float and double

A floating-point variable is capable of storing a number with a fractional part. The C++ language supports three different sizes of floating-point: float, double and long double.

Type	Size (bytes)	Min value	Max value	Sample data (To Store)
float	4	-3.4 E 38	3.4 E 38	Average
double	8	-1.7 E 308	1.7 E 308	Huge fractional calculation
long double	10	-3.4 E 4932	3.4 E 4932	Stellar distances

Data Type: char

A char variable is capable of storing any character on the keyboard. The ASCII code of the character is stored.eg the symbol '+', the letter 'A' etc.

Type	Size (bytes)	Domain	Sample data (To Store)
char	1	Any character on the keyboard	Symbol operator

Data Type: string

A string is a sequence of characters used to store text, such as words, sentences, or symbols.

Type	Size (bytes)	Domain	Sample data (To Store)
string	Variable	A sequence of characters	Student name

Note: To use string, include the `<string>` library:

```
#include <string>
using namespace std;
int main() {
    string name = "Alice";
    string message = "Hello, world!";
    return 0;
}
```

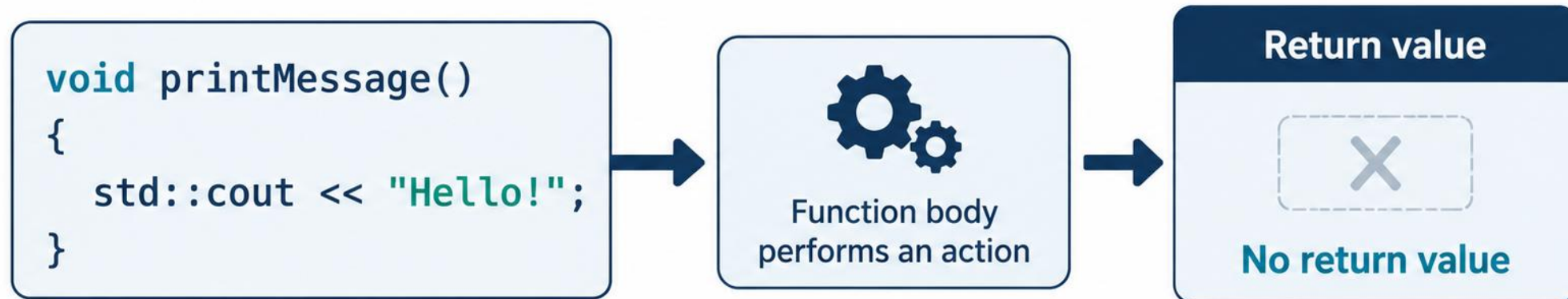
Data Type: bool

The bool type in C++ is a built-in logical data type that can only take two values: true (usually 1) and false (0).

Type	Size (bytes)	Domain	Sample data (To Store)
bool	1	True or False	Passes exam

Data Type: void

The void type has no values and no operations. In other words, both the set of values and the set of operations are empty. Although this might seem unusual, we will see later that it is a very useful data type.



Variable declarations

A variable declaration specifies the type and the name of the variable.

A variable has a type and it can contain only values of that type.

For example, a variable of the type `int` can only hold integer values.

Syntax:

```
data type identifier;           //declaration
```

```
data type identifier = value;  // initialization
```

Examples:

```
int age;                        //declaration
```

```
int no_of_books = 45;          //initialization
```

```
char letter = 'y';
```

```
string name = "Alice";
```

```
double price = 2493.14;
```

```
float temp = -24.5;
```

Constant declarations

Constants are used to store values that never change during the program execution.

Using constants makes programs more readable and maintainable.

Syntax:

```
const data type identifier = value;
```

Examples:

```
const double phi_number = 3.14;
```

```
const double rate = 7.8;
```

```
const int x = 45;
```

Operators and Expressions



KAZAKH-BRITISH
TECHNICAL
UNIVERSITY

Programs use data stored in variables and perform different types of operations on that data.

The data on which operations are performed are known as operands and the types of the operations performed on them are known as operators.

The combination of operators and expressions are known as expressions

Consider the following c++ statement: $z * y$

- z and y are the operands

- $*$ (multiplication is the operator)

- $z * y$ is an expression

Types of Operators

	Operator	Type
Unary Operator →	++, --	Unary Operator
Binary Operator {	+, -, *, /, %	Arithmetic Operator
	<, <=, >=, ==, !=	Relational Operator
	&&, , !	Logical Operator
	&, , <<, >>, ~, ^	Bitwise Operator
	=, +=, -=, *=, /=, %=	Assignment Operator
Ternary Operator →	?:	Ternary or Conditional Operator

Arithmetic operators

These operators are used to perform simple arithmetic calculations like addition, subtraction, multiplication and division. All the arithmetic operators are binary in nature. Only the "-" operator can be both unary or binary

```
#include <iostream>
using namespace std;
int main()
{
    int a, b;
    a = 40, b = 20;
    cout << "\n Sum of a and b:" << a+b;
    cout << "\n Difference of a and b:" << a-b;
    cout << "\n Product of a and b:" << a*b;
    cout << "\n Quotient of a and b:" << a/b;
    cout << endl;
    return 0;
}
```

Sum of a and b:60
Difference of a and b:20
Product of a and b:800
Quotient of a and b:2

Arithmetic operators

The division operator (/) does integer division only. This means if we divide say 45 by 6, we will get the answer as 7 and not 7.5 as expected. This is because the / operator discards the decimal part after division.

To get the accurate answer in the above case we must write 45.0/6.0.

```
#include <iostream>
using namespace std;
int main()
{
    cout << "Divide One:" << 45 / 6 << endl;
    cout << "Divide Two:" << 45.0 / 6.0 << endl;
    cout << "Divide Three:" << 45 / 6.0 << endl;

    int c;
    c = 45.0 / 6.0;
    cout << "Divide c:" << c << endl;
    return 0;
}
```

```
Divide One:7
Divide Two:7.5
Divide Three:7.5
Divide c:7
```

Remainder (%)

The modulus operator represented by the percentage sign (%) gives the remainder of division. The modulus operator can be used only with integers.

```
#include <iostream>
using namespace std;
int main()
{
    int a, b;
    a = 5, b = 2;
    cout << "\n Quotient of a and b:" << a / b;
    cout << "\n Remainder of division is:" << a % b;
    return 0;
}
```

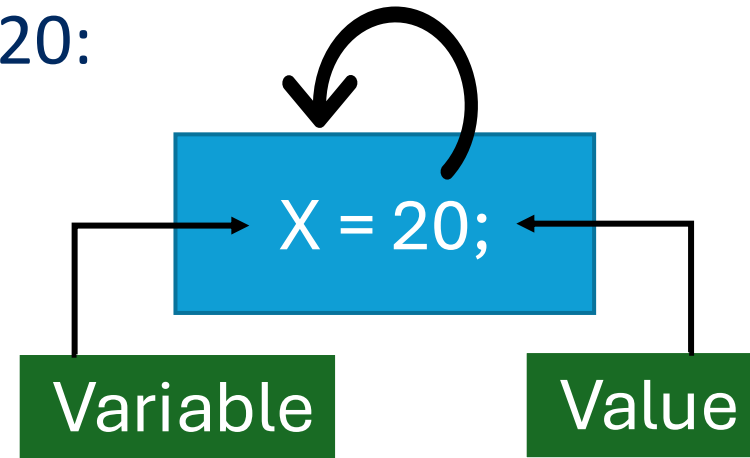
$$5 \div 2 = 2 \text{ remainder } 1$$

$$\text{or } 5 = 2 \times 2 + 1$$

Quotient of a and b:2
Remainder of division is:1

Assignment (=) Operator

The Assignment operator is used to assign a value to a variable.
In the example the variable x is assigned a value 20:



The variable is always on the left hand side
and the value on the right hand side.

The value can be another variable or an expression.

For eg:

a = b;	the value stored in variable b is assigned to a
p = q+r;	the sum of q and r is stored in the variable p

C++ Shorthands

These are operators used to perform an arithmetic function on an operand and assign the new value to the operand at the same time.

Operator	Description	Example	Equivalent to:
<code>+=</code>	Addition assignment	<code>A+=B</code>	<code>A=A+B</code>
<code>-=</code>	Subtraction assignment	<code>A -=B</code>	<code>A=A-B</code>
<code>*=</code>	Multiplication assignment	<code>A*=B</code>	<code>A=A*B</code>
<code>/=</code>	Division assignment	<code>A/=B</code>	<code>A=A/B</code>
<code>%=</code>	Modulus assignment	<code>A%=B</code>	<code>A=A%B</code>

C++ Shorthands

The increment (++) / decrement (--) operator increases / decreases the value of a variable by 1.

The syntax is:

variable ++;

variable --;

a++;

b--;

++ variable;

-- variable;

is the same as

is the same as

Examples: `int a = 2, b;`

`a=a+1;`

`b=b-1;`

`a++;`

`++a;`

`b = --a;`

The increment / decrement operators do not work on constants.

Thus: `6++;` // gives error as 6 is a constant

`x = --9;` // gives error as 9 is a constant

ASCII Table



KAZAKH-BRITISH
TECHNICAL
UNIVERSITY

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Math Functions

C++ provides many built-in mathematical functions through the `<cmath>` library.

Function	Description	Example
<code>sqrt(x)</code>	Calculates the square root of x	<code>sqrt(25) → 5</code>
<code>abs(x)</code>	Returns the absolute value of x	<code>abs(-7) → 7</code>
<code>sin(x)</code>	Calculates the sine of an angle	<code>sin(x)</code>
<code>max(a, b)</code>	Returns the larger value	<code>max(5, 8) → 8</code>
<code>min(a, b)</code>	Returns the smaller value	<code>min(5, 8) → 5</code>
<code>pow(x, y)</code>	Calculates x^y	<code>pow(2, 3) → 8</code>

Math Functions

```
#include <iostream>
#include <cmath>
#include <algorithm>
using namespace std;
int main() {
    double x = 25;
    cout << sqrt(x);           // 5
    cout << abs(-10);          // 10
    cout << pow(2, 3);         // 8
    cout << sin(x);            // sine of x
    cout << max(5, 8);         // 8
    cout << min(5, 8);         // 5
    return 0;
}
```

Note: Mathematical functions such as `sqrt()`, `abs()`, `sin()`, and `pow()` are provided by the `<cmath>` library, while `min()` and `max()` are provided by the `<algorithm>` library.

Comments

Comments are notes written in the source code to explain what the code does. They are ignored by the compiler and do not affect how the program runs.

Single-line comment

```
// Calculate the total price  
int total = price * quantity;
```

Multi-line comment

```
/*  
Calculate the total price  
for all items  
*/  
int total = price * quantity;
```



KAZAKH-BRITISH
TECHNICAL
UNIVERSITY

Questions?

Lecture 1

z.bolatov@kbtu.kz



Zhiger Bolatov

Senior Lecturer
School of IT&I